COURSE METHOD

Course Method: Spiral + Ship

منهج الكورس: تعلّم حلزوني وانشر باستمرار

Orientation

No code changes

How we will learn and ship

LEARNING OUTCOME

Understand spiral learning, vertical slices, frequent shipping, and why architecture appears after real product pain instead of before it.

There are two exhausting ways to learn product engineering.

Either study everything before building, or rush features without ever returning to improve the foundation.

EXTREME 1

Theory-first forever

- Weeks of abstractions before users see value.
- Patterns memorized without understanding the pain they solve.
- Deployment postponed until it becomes intimidating.

EXTREME 2

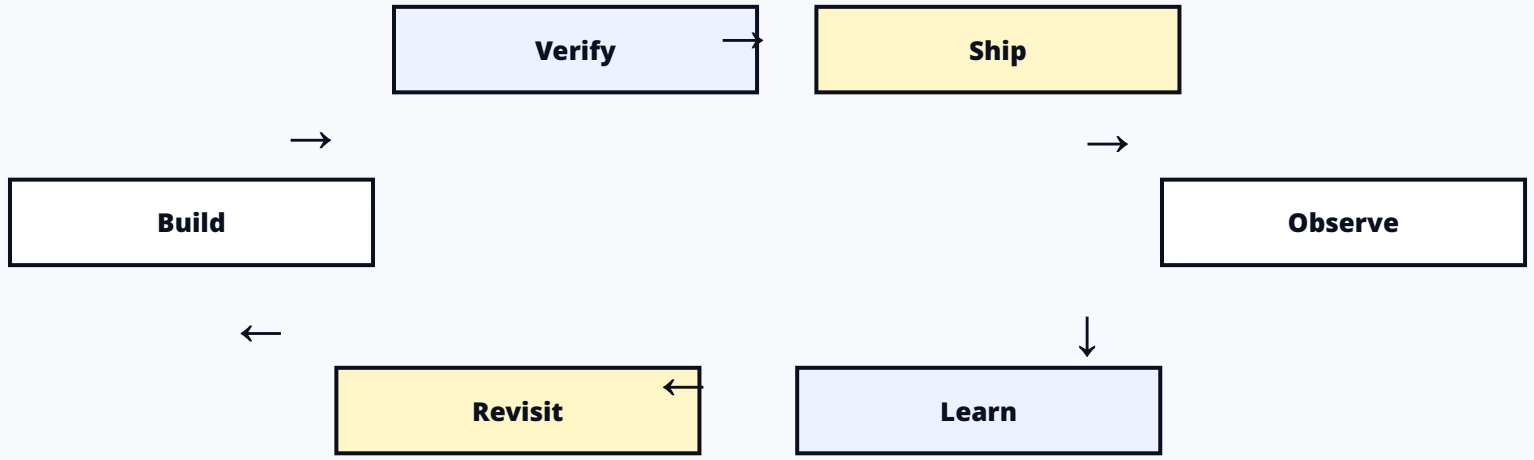
Feature rush forever

- Every shortcut becomes permanent.
- Boundaries blur as features pile up.
- The product works until one failure exposes everything at once.

This course uses a third path: build a small working slice, ship it, observe what becomes painful, then revisit the same idea with more depth.

Meet the idea simply. Use it. Return when the problem gets richer.

The second visit is not repeated content - the product has changed, so the question has changed.



Deployment spiral

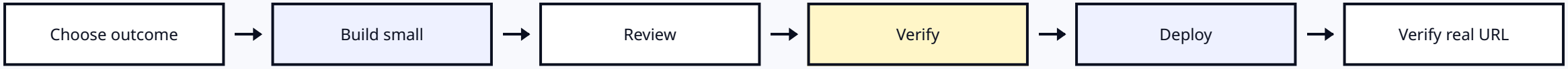
First: put one page online. Later: environment variables, domains, staging, Docker builds, and safer release flow.

Authentication spiral

First: sign up and login. Later: session-derived ownership, admin role protection, and stronger security concerns.

Shipping is not reckless pushing.

A shipped increment is small, coherent, verified, and available in the environment where it is supposed to work.



Surface build issues early

Environment, build, and runtime problems appear while the project is still small.

Think beyond localhost

Every future feature is designed with real deployment behavior in mind.

Normalize release

Deployment stops being a frightening final chapter.

Day 1 deploys on purpose: deployment is part of development from the first day, not a graduation ceremony at the end.

Build one thin journey through the necessary layers.

The useful unit is end-to-end user value - not weeks of disconnected layers.

User need	UI	Server rule	Data / provider	Deployed verification
What outcome matters?	What does the user see or do?	What trusted behavior must happen?	What persistence or external work is needed?	Does the real journey work?

JOBS SLICE

Display a real list

Connect a data source to a public page and verify the result end to end.

APPLY SLICE

Accept one real submission

Move through UI, server rules, storage, persistence, and deployed verification.

Why small slices: they are easier to review, expose integration problems earlier, and produce visible progress.

Thin does not mean fake; it means bounded.

Architecture becomes meaningful when you can feel the problem it solves.

Before pain, a pattern can look like ritual. After pain, its benefit and cost become obvious.

Pattern	Pain that triggers it	Why now
Repository layer	Database details leak into business logic.	The boundary becomes worth paying for.
Background worker	Synchronous AI creates slow requests, timeouts, or burst pressure.	Latency and reliability pain are now visible.
Caching	Repeated or slow work is measured after correctness exists.	Evidence justifies the added complexity.

The course rule: introduce the pattern after the pain appears.

> **Production-first does not mean "maximum architecture first."** It means the smallest safe step with explicit boundaries and honest deferral.

One capability can mature through multiple honest passes.

Earlier steps are not “wrong” when they clearly state what they do and do not guarantee.

PASS 1

Fake upload UI

Candidate sees where a resume will attach.

Boundary: no claim of safe storage yet.

PASS 2

Real private storage

Server-proxied upload, controlled metadata, and temporary admin access.

Boundary: production-realistic file handling.

PASS 3

Synchronous AI screening

Save the application first, then structured screening in the request. Record COMPLETED / FAILED.

Boundary: proves the flow; not the final scale architecture.

PASS 4

Queued worker - Day 16

QStash-backed processing with claims, retries, and reconciliation.

Boundary: addresses latency and reliability pain.

Critical production habit: application submission succeeds even when optional AI screening fails.

The workflow is repeatable even when the feature changes.

The course teaches a routine for turning a product outcome into verified, shipped evidence.

1. Choose outcome
One product or operational result.

2. Focus work
Feature branch once that workflow is introduced.

3. Implement
Smallest production-realistic slice.

4. Review
Correctness, scope, and intended behavior.

5. Verify locally
Types, lint/build, and real behavior.

6. Deploy
Use the appropriate environment.

7. Verify deployed
Test the actual user journey.

8. Capture decisions
Known limits and what comes next.

Branch

Isolates focused work.

Review

Checks the change against the intended outcome.

Verification

Collects evidence instead of assuming success.

Formal staging + branch rules arrive on Day 7.

Before adding complexity, pass three tests.

Simple is allowed. Unexplained risk is not.

01

Does this create real value or necessary foundation?

If it does not move a user journey or operational capability forward, it may not belong yet.

02

Does it acknowledge the most relevant current risk?

Do not ignore the failure mode that matters most at the product's current stage.

03

Can we explain what is intentionally deferred?

Deferred work should have a reason and a trigger for revisiting it.

NOT NOW PARKING LOT

- **Queue**
- **Advanced caching**
- **Multi-region deployment**
- **Separate backend**

Deferred is not forgotten. We revisit these choices when product evidence changes.

The same product concerns keep returning at greater depth.

This is the roadmap pattern you should notice as the course progresses.

Deployment

One page online → custom domains → staging → runtime config → release safety.

Authentication

Sign-up/login → session ownership → admin role protection.

Resume handling

Placeholder UI → private DigitalOcean Spaces → temporary admin access.

AI screening

Synchronous OpenAI flow → QStash worker with retries + reconciliation.

Architecture

One modular monolith → clearer internal boundaries as pain appears.

What stays stable

One product, one deployment at this stage, and architecture that protects responsibilities.

What evolves

Depth, operational controls, and patterns increase as evidence justifies them.

Mindset: do not judge an early pass using the requirements of a later pass. Judge whether it is honest, safe enough for now, and easy to evolve.

Lecture 4 explains the method - not the implementation details.

The boundaries matter because future lectures will introduce these technologies only when the product needs them.

NOT NOW

Git commands

Branch rules and staging workflow arrive later.

NOT NOW

Queue + worker APIs

The worker comes when synchronous AI produces real pain.

NOT NOW

Storage + auth internals

Those capabilities are taught inside the slices that need them.

Do not confuse production-first with everything-at-once.

Also out of scope: calling synchronous AI the final scalable design, forcing a deploy after broken verification, or treating microservices as automatically more advanced than a monolith.

Can you explain how the course will teach before Day 1 starts?

Lecture 4 is complete when the method feels like an engineering loop rather than a syllabus trick.

- [ ] Explain spiral learning with deployment or authentication.
- [ ] Describe a vertical slice and give a wazifa.app example.
- [ ] Name all four upload / AI progression stages.
- [ ] Define Ship as release plus verification, not reckless pushing.
- [ ] State the “patterns after pain” rule with an example.
- [ ] Outline the branch → review → verify → deploy → observe loop.

01

Meet an idea simply, use it, then revisit it with real context.

02

Ship small vertical slices and verify the real environment.

03

Introduce architecture after the pain makes its value visible.

NEXT LECTURE

Lecture 5 - Project Overview